

ACTIVIDAD PRÁCTICA: TELEMETRÍA Y TELECONTROL

Estudiante

Victoria Etcheverry Senas | victoria.etccheverry@estudiantes.utec.edu.uy | 5.515.176-1

UTEC: Instituto Regional Sur Oeste

Fray Bentos – Rio Negro - Uruguay

Fecha de realización

26/04/2025

Desarrollo

1. Opción 1: Sistema de Refrigeración Industrial

Tabla 1. Pinout actividad 1.

Fuente: Elaboración Propia.

Sistema	
LED	D2 (ESP8266)
Pin 1 (Potenciómetro)	VCC (3.3V)
Pin 2 (Potenciómetro)	A0 (ESP8266)
Pin 3 (Potenciómetro)	GND

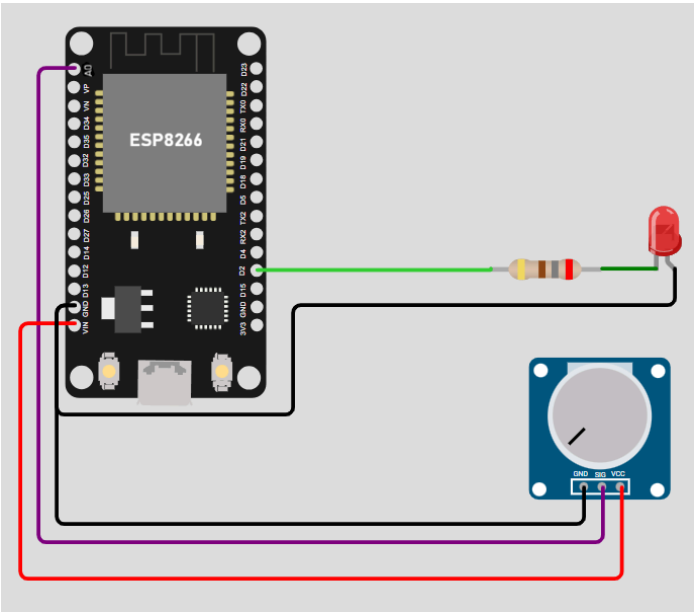


Fig.1 Diagrama de conexiones del sistema.

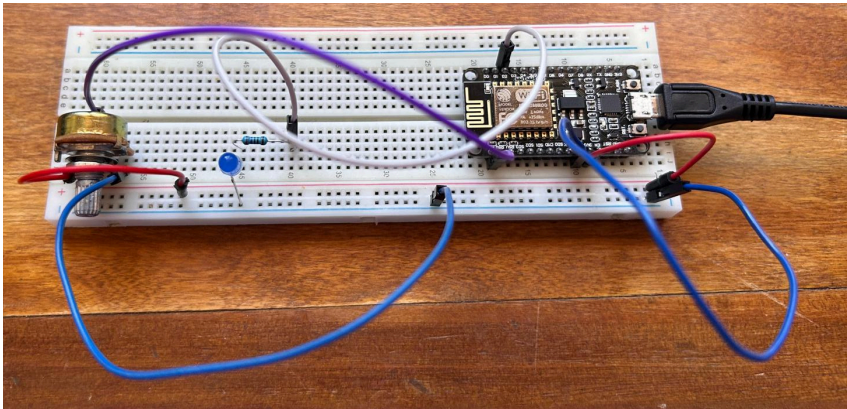


Fig.2 Conexión del sistema Actividad 1.

Tabla 2. Tópicos.

Fuente: Elaboración Propia.

Tópico	Tipo	Descripción
refrigeracion/motor1/temperatura	Publicación (ESP - Broker)	Envía el valor de temperatura simulado (0–100 °C) leído desde el potenciómetro cada 9 segundos.
refrigeracion/motor1/comando	Suscripción (Broker - ESP)	Recibe comandos del usuario. Los mensajes posibles son ON (encender ventilador) y OFF (apagar ventilador).
refrigeracion/motor1/estado	Publicación (ESP - Broker)	Informa el estado del ventilador cuando cambia. Pública VENTILADOR_ON o VENTILADOR_OFF.

Se definió una estructura jerárquica de tópicos basada en el sistema físico (refrigeración) y el dispositivo controlado (motor1).

Esto permite una organización clara y escalable, facilitando la identificación de cada variable y función dentro del sistema.

Se separaron los tópicos de telemetría (temperatura), telecontrol (comando) y estado, lo que mejora la legibilidad y mantenimiento del sistema.

Apéndices

Apéndice A. Videos

Link: https://drive.google.com/drive/folders/1GZf2Mdvu2SQJEmCI_pcp_QjdVWArZpme?usp=sharing

Apéndice B. Código

```
#include <ESP8266WiFi.h> // Librería para conexión WiFi
#include <PubSubClient.h> // Librería para MQTT

// WiFi
const char* ssid = "Victoria-2.4GHz"; // Nombre de la red que usamos
const char* password = "vitoetchevery"; // Contraseña
// Broker
const char* mqtt_server = "192.168.1.69"; // IP del borker (compu con mosquitto)

// Creación de cliente WiFi y cliente MQTT
WiFiClient espClient;
PubSubClient client(espClient);

// Pines
#define LED_PIN D2 // Pin LED (simula ventilador)
#define POT_PIN A0 // Entrada analógica (potenciómetro)

// Variables
unsigned long lastMsg = 0; // Guarda el último tiempo de envío
const long intervalo = 9000; // Intervalo de envío (9 segundos)
bool estadoLED = false; // Estado actual del LED

// WiFi
void setup_wifi() {
  Serial.begin(115200); // Inicializa comunicación serial
  WiFi.mode(WIFI_STA); // Modo estación (cliente)
  WiFi.begin(ssid, password); // Conexión a la red

  Serial.print("Conectando WiFi...");

  // Espera hasta conectar
  while (WiFi.status() != WL_CONNECTED) {
```

```

    delay(500);
    Serial.print(".");
}

Serial.println("\nWiFi conectado");
Serial.print("IP: ");
Serial.println(WiFi.localIP()); // Muestra IP asignada
}

// Callback MQTT
// Se ejecuta automáticamente cuando llega un mensaje al tópico suscripto
void callback(char* topic, byte* payload, unsigned int length) {

    String mensaje = "";
    // Conversión del mensaje recibido (byte) a String
    for (int i = 0; i < length; i++) {
        mensaje += (char)payload[i];
    }

    Serial.print("Mensaje recibido: ");
    Serial.println(mensaje);

    // Control del LED según comando recibido
    if (mensaje == "ON" && !estadoLED) {
        digitalWrite(LED_PIN, HIGH); // Enciende LED
        estadoLED = true;

        // Publica estado (desafío extra)
        client.publish("refrigeracion/motor1/estado", "VENTILADOR_ON");
    }

    else if (mensaje == "OFF" && estadoLED) {
        digitalWrite(LED_PIN, LOW); // Apaga LED
        estadoLED = false;

        // Publica estado (desafío extra)
        client.publish("refrigeracion/motor1/estado", "VENTILADOR_OFF");
    }
}

// Reconexión MQTT
// Intenta reconectar si se pierde la conexión con el broker
void reconnect() {
    while (!client.connected()) {
        Serial.print("Conectando a MQTT...");

        if (client.connect("ESP8266Client")) {
            Serial.println("conectado");

            // Se suscribe al tópico de comandos
            client.subscribe("refrigeracion/motor1/comando");

        } else {
            Serial.print("error, rc=");
            Serial.print(client.state()); // Código de error
            Serial.println(" reintentando en 5s");
            delay(5000);
        }
    }
}

void setup() {

```

```
pinMode(LED_PIN, OUTPUT); // Configura LED como salida
digitalWrite(LED_PIN, LOW); // Inicialmente apagado

setup_wifi(); // Conexión a WiFi

client.setServer(mqtt_server, 1883); // Configura broker MQTT
client.setCallback(callback); // Asocia función callback
}

void loop() {

    // Verifica conexión MQTT
    if (!client.connected()) {
        reconnect();
    }
    client.loop(); // Mantiene comunicación MQTT

    unsigned long now = millis();

    // Envío de temperatura cada 9 segundos
    if (now - lastMsg > intervalo) {
        lastMsg = now;

        int valorADC = analogRead(POT_PIN); // Lectura analógica

        // Escalado de 0–1023 a 0–100 °C
        float temperatura = (valorADC / 1023.0) * 100.0;

        // Conversión a string para enviar por MQTT
        char tempString[10];
        dtostrf(temperatura, 4, 2, tempString);

        Serial.print("Temperatura: ");
        Serial.println(tempString);

        // Publicación en tópico de telemetría
        client.publish("refrigeracion/motor1/temperatura", tempString);
    }
}
```
